Autonomous Support Investigation Agent

Design document · architecture, techniques and capabilities

A locally runnable LangGraph agent that investigates new customer-support tickets using ~28K historical English tickets. It retrieves and compares past cases, decides which tools to use, loads skills on demand, asks the customer for clarification when needed, verifies its own answer against the evidence, lets a support lead correct it with their own diagnosis, and adds accepted resolutions back into its knowledge base.

23,608	10	7	4	7
historical tickets indexed	graph nodes	LLM-callable tools	dynamic skills	LangSmith evaluators

Layer	Choice
Orchestration	LangGraph 1.2 · StateGraph · SqliteSaver checkpointer · SqliteStore · interrupt / Command(resume)
LLM (cloud)	Groq · openai/gpt-oss-120b (investigation, response) + openai/gpt-oss-20b (triage, judge, memory)
Vector database	Qdrant Cloud (free tier, strict mode) or embedded local mode · named dense + sparse vectors · RRF fusion
Embeddings (local)	fastembed · BAAI/bge-small-en-v1.5 (384-d dense) + Qdrant/bm25 (sparse)
Structured storage	SQLite · app.db (historical tickets, cases, event log, LLM usage) · checkpoints.db · memory.db
Human in the loop	Clarification questions to the customer · support-lead review: accept, amend with own root cause + solution, or reject
Interfaces	FastAPI REST + Server-Sent Events · single-page web UI · CLI · demo script
Evaluation	LangSmith evaluate() with 7 evaluators on a 40-ticket held-out split

Prepared for: DCKAP AI Engineer Assessment (Level 2) · Date: 6 November 2026 · Revised for the support-lead knowledge feature, the lead-solution API and the demo reset script.

Contents

1. Executive summary . . . 4
What the agent can do . . . 4
2. Requirements coverage . . . 5
Expected agent behaviour (brief §4) mapped to the graph . . . 6
3. System architecture . . . 6
Components . . . 6
Project layout . . . 7
4. Data and ingestion . . . 7
Files taken from Hugging Face . . . 7
Pipeline (scripts/ingest.py) . . . 7
5. LangGraph workflow design . . . 8
Pattern and rationale . . . 8
Nodes . . . 9
Routing . . . 10
Loops, interrupts and parallelism . . . 10
6. Lean design: from 17 nodes to 10 . . . 11
7. State and memory . . . 11
Graph state . . . 11
Short-term memory . . . 12
Long-term memory . . . 12
Customer history . . . 12
Storage map . . . 12
8. Case-based RAG . . . 13
9. Tools and skills . . . 13
Tools (the model decides if, when and which) . . . 13
Skills (progressive disclosure) . . . 14
10. Response, evidence check and confidence . . . 14
respond . . . 14
evaluate . . . 14
11. Continuously growing case knowledge . . . 15
Support-lead knowledge (lead-verified cases) . . . 15
12. LLM layer, usage accounting and limits . . . 16
Token-efficiency techniques . . . 16
Structured output . . . 17
Measured cost per ticket . . . 17

13. Evaluation (LangSmith) . . . 17

14. API, streaming and frontend . . . 18

Web UI features . . . 18

15. Techniques catalogue . . . 19

16. Verified demo runs . . . 20

Behaviour observed during testing . . . 20

17. How to run . . . 21

18. Limitations and next steps . . . 21

Appendix · Configuration reference . . . 22

1. Executive summary

The agent turns the brief's 9-step behaviour into a **ReAct investigation loop wrapped in a verify-and-commit pipeline**. Steps every ticket needs (persist, load memory, retrieve, triage, verify, persist, index) are fixed nodes, so they are predictable and auditable. The open-ended part (which evidence to gather, whether to ask the customer, whether to escalate) is decided by the model inside a tool loop.

Every design choice was made against two real constraints that were **measured before anything was loaded or tested**: the Groq free tier of the time (8K tokens/minute and 200K tokens/day per model) and Qdrant Cloud's strict mode (filters on payload fields that have no index are rejected). The result is a lean 10-node graph that uses one fewer LLM call per ticket than the first version. The Groq account has since moved to a paid plan: usage is recorded per key and model, and no client-side limits are enforced.

The review step is also a **knowledge channel**. When the historical base has no answer, the support lead can enter the real root cause and fix. The reply is rewritten from it and checked again, and the case is indexed as **lead-verified**, so similar future tickets get the lead's fix (section 11). This added no node: it reuses review → respond → evaluate.

> **Verified on real services.** Normal resolution, clarification with pause and resume, long-term memory reuse and retrieval of a newly resolved case were all demonstrated end to end on Groq and Qdrant Cloud (section 16). After the token trims a resolved ticket costs about 7.1–10.6K tokens in 5–6 Groq calls, about 25% less input than before.

What the agent can do

Ability	What it does
Intake	Persists every new query as an **open case** before doing anything else, and re-opens it for follow-up turns.
Context	Loads the customer's **long-term memory** (durable facts) and **case history**, including still-open cases.
Retrieval	**Hybrid search** (meaning + keywords) over 23,608 historical tickets plus accepted new cases, with the query enriched by remembered facts. Lead-verified cases rank first among equally similar ones.
Triage	Classifies queue, type, priority and tags using the ticket and the labels of similar past cases; output is normalised to the dataset's vocabulary.
Skills	Picks 1–2 investigation **skills** from a catalog and loads only their full instructions (progressive disclosure).
Investigation	Decides **by itself** which tools to call: targeted filtered searches, opening full records, customer history, category statistics, re-classification. Several tools per round, budgeted.
Clarification	Asks the customer **one focused question** when information only they have is missing; the run pauses, survives restarts and resumes with the answer. No fixed limit, but repeated questions, questions after "no more details" and requests for secrets are blocked.
Evidence reasoning	Compares resolutions across **multiple cases**, reports consensus and conflicts, and never just copies the nearest answer.
Answer or escalate	Writes a grounded customer reply with cited cases, or an **escalation hand-off** (target team, urgency, internal summary, holding reply).
Self-verification	Checks its own draft (rules + an independent LLM judge), scores **confidence**, retries with a targeted search, and escalates instead of sending an unsupported answer.
Human trust gate	A support lead **accepts**, **rejects** with feedback, or **amends** with their own root cause and solution before the answer becomes knowledge (auto-accept mode for demos/evals).
Support-lead knowledge	The lead's diagnosis becomes authoritative evidence (LEAD-<case>), the reply is rewritten from it and re-checked, and the case is indexed as **lead-verified**. Also works on finished or escalated cases.
Growing knowledge	Indexes accepted, verified resolutions and lead-diagnosed cases into Qdrant so **future tickets retrieve them**.
Learning about customers	Extracts durable customer facts, updates or forgets them, blocks secrets and payment data, and caps memory size.
Multi-turn	Handles **follow-up messages** in the same thread with the full prior context; a follow-up sent while a question is open is taken as its answer.

Ability	What it does
Usage-aware	Records Groq tokens per key, model and day (paid plan: no client-side limits), throttles and retries Qdrant calls, and resumes a failed run from its checkpoint.
Observable	Streams every step with token counts, persists an event log for replay, and supports LangSmith tracing and evaluation.

2. Requirements coverage

Each requirement from the assessment brief and where it is implemented.

Requirement (brief)	Implementation	Where
A. LangGraph orchestration: multiple nodes	10-node StateGraph	app/graph.py
Conditional routing	5 routers: after intake, after investigate, after tools, after evaluate, after review	app/nodes.py
At least one meaningful loop / retry	4 loops: tool loop · evidence-check retry · review rejection · lead amend (review → respond)	graph.py
Interrupt / resume for more information	interrupt() inside the ask_customer tool and in review (accept / amend / reject); Command(resume=...)	tools.py, nodes.py
B. Index tickets in a vector DB	Qdrant Cloud, named dense (bge-small) + sparse (BM25) vectors	vectorstore.py
Use metadata (queue, type, priority, tags)	Payload filters chosen by the model + strict-mode payload indexes; neighbour label votes for triage	vectorstore.py, labels.py
Compare evidence across cases, handle conflicts	Prompts require consensus vs conflicts; ≥2 cited cases (or the lead's diagnosis) enforced by the evidence check	nodes.py
Follow-up / targeted retrieval	search_similar_tickets with filters, chosen by the model; judge-suggested search on retry	tools.py
C. Short-term memory	SqliteSaver checkpointer per thread_id; follow-ups keep messages, evidence, clarifications, lead guidance	graph.py
Long-term customer memory, decide what to keep	SqliteStore namespace per customer; LLM memory policy + PII filter + cap	memory.py
D. ≥4 meaningful tools, model decides	7 tools bound to the model; no fixed sequence	tools.py
≥3 reusable skills, loaded dynamically	4 markdown skills; catalog shown, full text loaded only when selected	skills/
E. LangSmith evaluators + datasets	7 evaluators (rules + LLM judges), held-out dataset, evaluate()	evals/
3. Store every query; store final resolution; make it retrievable	SQLite cases table from intake; finalize stores the result; accepted cases indexed (source=live, lead_verified when the lead diagnosed them)	db.py, nodes.py
New/open cases used as customer history	customer_cases() reads all statuses; shown to the model; citable as evidence	db.py, nodes.py
5. Output: classification, evidence, analysis, resolution, escalation, confidence	finalize builds exactly these fields (plus evaluation, clarifications, skills, review, lead guidance)	nodes.finalize
6. Local, Python + LangGraph, cloud LLM allowed, dataset-only knowledge, secrets outside VCS	Runs locally; Groq cloud LLM; only the dataset (and the lead's own input) is knowledge; .env is gitignored	config.py, .gitignore
7. Code + deps + README; ingest script; demo of 4 paths	requirements.txt, README.md, scripts/ingest.py, sync_qdrant.py, demo.py (4 required paths + a lead scenario), reset_live_cases.py	repo
8. Parallel execution (plus)	finalize || update_memory fan-out; ToolNode runs a round's tool calls in parallel	graph.py
Human expertise as knowledge (beyond the brief)	Lead's root cause + solution become authoritative evidence and lead-verified cases	nodes.review_update, finalize

Expected agent behaviour (brief §4) mapped to the graph

#	Brief step	Graph
1	Receive ticket, persist as open case	intake
2	Load short-term state and long-term memory	checkpointer (thread) + intake (store, history)
3	Classify / triage with ticket and historical evidence	retrieve → triage (neighbour label votes as prior)
4	Retrieve multiple cases, inspect previous resolutions	retrieve + search_similar_tickets / get_ticket_by_id
5	Use tools and relevant skills	investigate ↔ tools with dynamically loaded skills
6	Ask a focused clarification question and resume	ask_customer tool → interrupt → Command(resume)
7	Grounded resolution or escalation	respond
8	Evaluation / evidence check; if unsupported, investigate again	evaluate → investigate (retry) or → respond (escalate)
9	Persist final result; trusted cases searchable	review (accept / amend / reject) → finalize (persist + index) ‖ update_memory

3. System architecture

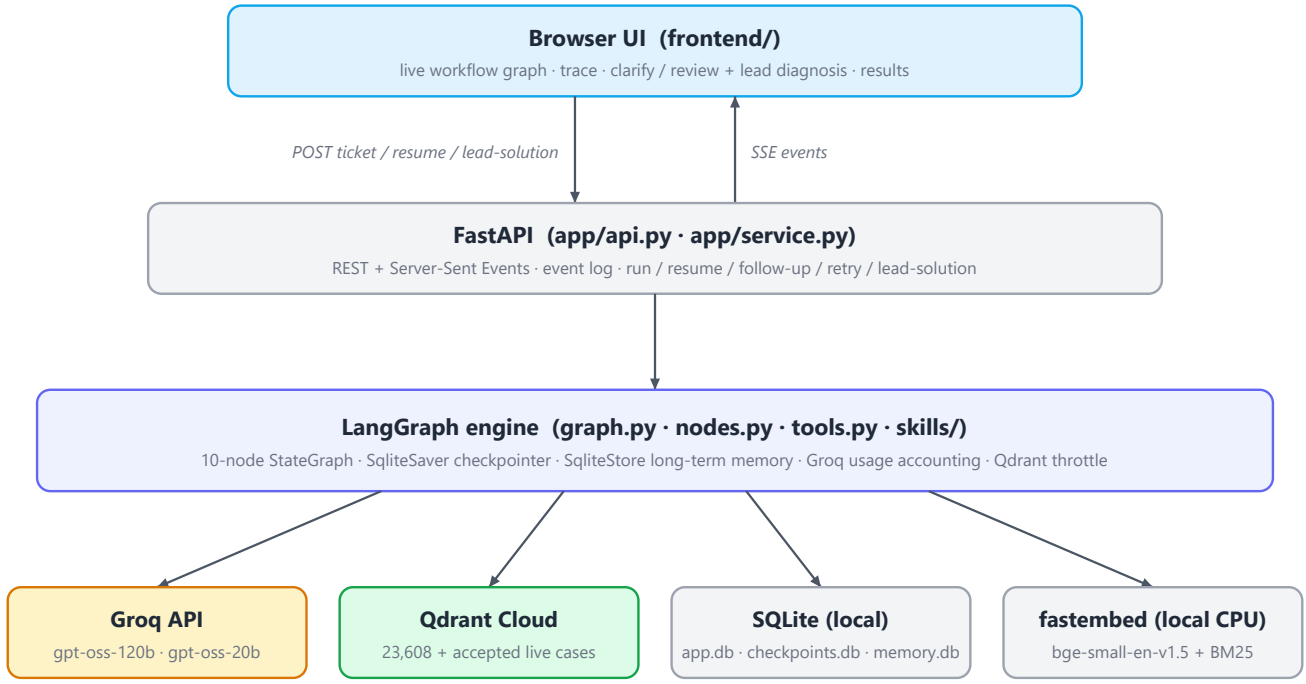


Figure 1 · Components and data flow. Embeddings are computed locally, so only the LLM is a paid cloud API.

Components

Component	Responsibility
frontend/ (index.html, app.js, styles.css)	Single page without a build step. Draws the real compiled graph (Mermaid), streams the trace, shows clarification and review panels (with the lead's root cause / solution fields), results, memory and case replay.
app/api.py	FastAPI endpoints incl. lead-solution; SSE streaming; startup ensures Qdrant payload indexes exist.
app/service.py	Runs / resumes the graph, applies a lead's solution to finished cases, converts LangGraph updates and custom events into UI events, persists them.
app/graph.py	Graph topology, SqliteSaver (short-term) and SqliteStore (long-term memory).
app/nodes.py	All node logic, prompts and pydantic output schemas; evidence check, confidence and the support-lead rules.

Component	Responsibility
app/tools.py	7 LLM-callable tools; state-changing tools return LangGraph Command updates; lead-safe answer clipping.
app/skills/*.md	4 conditional instruction modules + registry (catalog, render, default selection).
app/vectorstore.py	Qdrant hybrid search, upsert, payload indexes, throttled and retried calls.
app/llm.py + app/ratelimit.py	Groq models, structured output with local repair + fallback, token accounting (no Groq limits), Qdrant throttle.
app/memory.py	Long-term memory on the LangGraph store with PII guard and size cap.
app/db.py · app/labels.py	SQLite app.db (historical tickets, cases, events) · label vocabulary, normalisation, neighbour votes.
scripts/ · evals/	ingest, cloud sync, demo, CLI, reset of live cases · LangSmith evaluators and runner.

Project layout

```
backend/
  app/        api.py  service.py  graph.py  nodes.py  tools.py  state.py  llm.py  ratelimit.py
              vectorstore.py  memory.py  db.py  labels.py  samples.py  config.py  skills/*.md
  scripts/    ingest.py  sync_qdrant.py  demo.py  cli.py  reset_live_cases.py
  evals/      evaluators.py  run_evals.py
  data/       raw CSVs, app.db, checkpoints.db, memory.db, local qdrant/  (gitignored)
frontend/     index.html  app.js  styles.css
docs/         Support_Investigation_Agent_Design.pdf  build_design_pdf.py
.env          secrets (gitignored)      .env.example  template      README.md
```

4. Data and ingestion

Files taken from Hugging Face

Dataset: Tobi-Bueck/customer-support-tickets. Columns used: subject, body, answer, queue, type, priority, tag_1 … tag_8 (the language column is used only for filtering).

File	Rows	English rows used
aa_dataset-tickets-multi-lang-5-2-50-version.csv	28,587	16,338
dataset-tickets-multi-lang-4-20k.csv	20,000	11,923
dataset-tickets-german_normalized_50_5_2.csv	–	not used (German only)
Total English subset		**28,261** (the brief's ~28K)

Pipeline (scripts/ingest.py)

Step	Technique	Result
Download	huggingface_hub; skipped if files already exist	backend/data/raw/
Filter	language == en	28,261 rows
Clean	unescape literal \n; drop rows with body or answer ≤ 20 chars	–
De-duplicate	exact duplicates on (subject, body); the two files overlap heavily	23,648 unique
Hold-out	40 random tickets (seed 42) never indexed, used for evaluation	eval_holdout.jsonl
Store	SQLite historical_tickets (ids HT-00000…) for get_ticket_by_id and statistics	23,608 rows
Embed	dense: first 1,200 chars of subject+body, batch 32; sparse BM25 on full text	local CPU
Speed-up	records sorted by length so ONNX batches have little padding (≈ +40% throughput)	~58 min total
Index	named vectors dense + bm25 (IDF); deterministic uuid5 point ids = idempotent re-runs	23,608 points

Step	Technique	Result
Cloud copy	scripts/sync_qdrant.py copies vectors local → Qdrant Cloud without re-embedding	72 s, 0 retries

What is embedded and what is only stored. Only the problem text (subject + body) is embedded, because a new ticket describes a problem and is matched against past problems. Everything else rides in the Qdrant payload: case_id, subject, body, answer, queue, type, priority, tags, source (historical / live), and for live cases resolved_at, customer_id, lead_verified, root_cause and lead_solution. The past resolution is compared later, in the prompts.

5. LangGraph workflow design

Pattern and rationale

Support work is mostly a fixed procedure with one open-ended part. A single free-running agent is hard to control and audit; a fully fixed chain cannot adapt to vague tickets, conflicting evidence or escalations. The chosen pattern keeps the procedure deterministic and gives the model full control only where judgement is needed: inside the **investigate ↔ tools** loop. Two gates follow it: an automated **evidence check** and a human **trust gate**. When the lead supplies their own diagnosis at that gate, the run goes back to **respond**, not to investigate, because the lead has already given the answer. Figure 2 shows the whole graph.

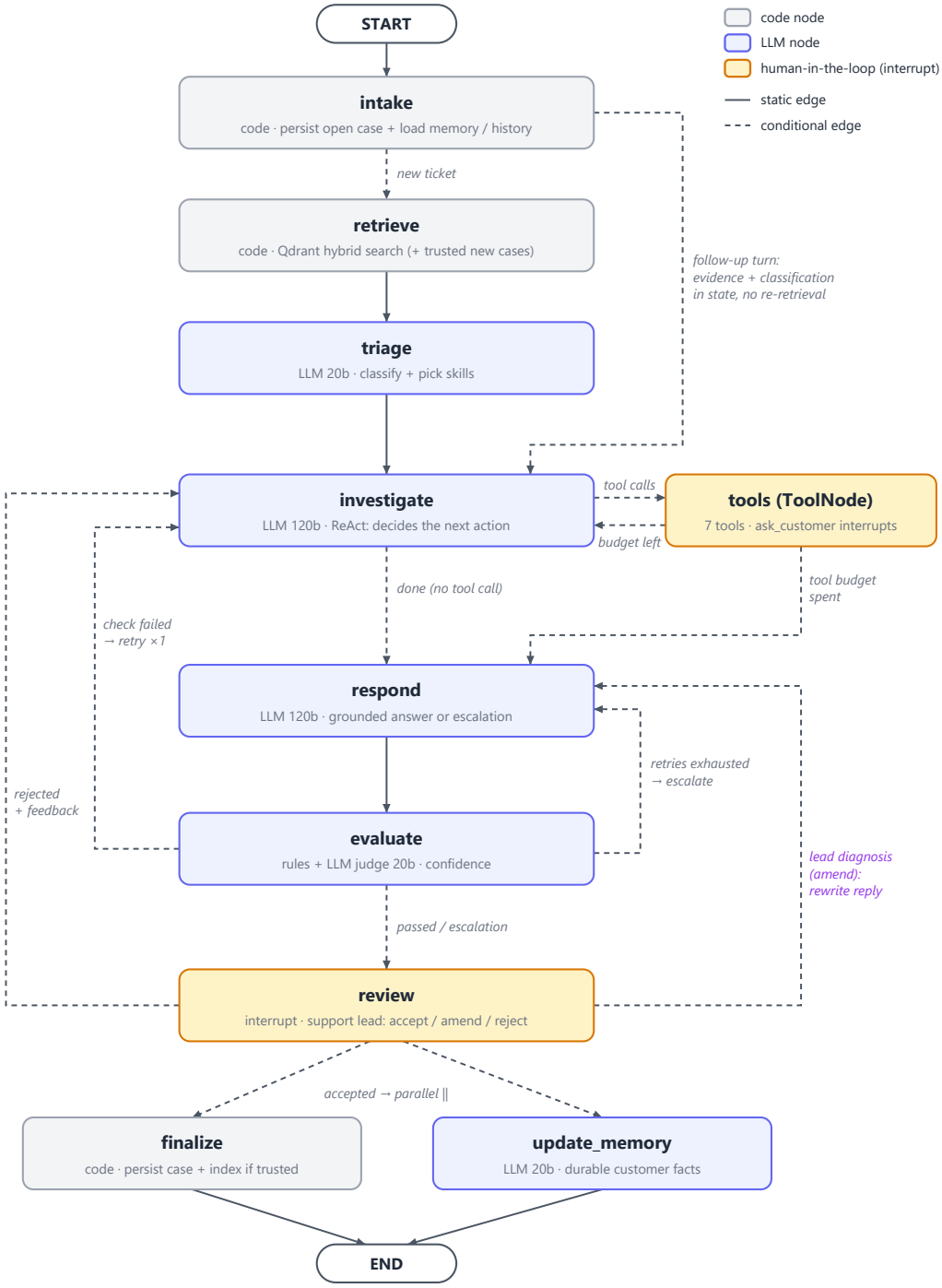


Figure 2 · The compiled 10-node graph. The UI draws this from the live graph object, so the picture always matches the code.

Nodes

Node	Kind / model	Responsibility
intake	code	Create the open case (or re-open it for a follow-up), reset per-turn fields, load long-term facts and the last 5 customer cases. A follow-up turn with evidence + classification in state routes straight to investigate.
retrieve	code	Hybrid search (k=8) with a memory-enriched query, no filters; if live cases exist, a dedicated source=live search for trusted new cases (similarity ≥ 0.6); neighbour label votes.
triage	LLM · 20b	Queue, type, priority, tags + 1–2 skills from the catalog. Sees only labels + subjects of the 6 nearest cases and compact label shares. Labels normalised; escalation skill added for high priority or repeat incidents.

Node	Kind / model	Responsibility
investigate	LLM · 120b	ReAct step with 7 tools bound. Starts from the evidence pool (top 5 cases, a pool-strength line) and searches only if it is weak. Gets every point the lead made on this case and must keep each one. Tool budget 2 rounds; escalation skill criteria only.
tools	ToolNode	Executes the round's tool calls (in parallel); tools update state through Command; ask_customer can interrupt.
respond	LLM · 120b	One schema for both outcomes: grounded resolution or escalation hand-off with holding reply. Top 5 cases, the ones the notes cite first; the lead's diagnosis and feedback verbatim when present; hand-off format loaded only when escalating.
evaluate	rules + LLM · 20b	Evidence check, confidence score, retry or forced escalation. The judge treats support-lead input as authoritative.
review	interrupt	Support lead: accept, amend (own root cause + solution) or reject with feedback; auto-accept configurable per run.
finalize	code	Build and persist the structured result; index the case in Qdrant if trusted (as lead-verified when the lead diagnosed it).
update_memory	LLM · 20b	Extract up to 4 durable customer facts per case; runs in parallel with finalize.

Routing

Router	Condition	Next
after intake	evidence pool and classification already in state (follow-up)	investigate · otherwise retrieve
after investigate	last AI message has tool calls	tools · otherwise respond
after tools	tool budget spent (rounds ≥ 2)	respond (skips a notes-only LLM call) · otherwise investigate
after evaluate	passed, or the draft is an escalation	review
	failed and retries left (1)	investigate with judge feedback and one targeted search round
	failed and retries exhausted	respond with forced escalation
after review	rejected	investigate with the lead's feedback
	amended (lead's root cause / solution)	respond: rewrite the reply from the diagnosis
	accepted	[finalize, update_memory] in parallel

Loops, interrupts and parallelism

- **Tool loop** (investigate ↔ tools): model-decided, capped at 2 rounds; asking the customer refunds its round so the answer can be used.
- **Evidence-check retry** (evaluate → investigate): one retry with the judge's feedback and suggested query, then escalation.
- **Review rejection** (review → investigate): the lead's feedback becomes a message (marked as the lead's, not the customer's); budgets reset and stale notes are cleared.
- **Lead amend** (review → respond): the diagnosis enters the evidence pool as LEAD-<case_id>; the reply is rewritten, evidence-checked and shown for review again.
- **Interrupts**: interrupt() inside the ask_customer tool (in the ToolNode) and in review. State is checkpointed, so a paused case survives a server restart; resumption uses Command(resume=…).
- **Parallelism**: a conditional edge returning [finalize, update_memory] fans out two independent side effects; ToolNode executes multiple tool calls of one round concurrently.

6. Lean design: from 17 nodes to 10

The first version mirrored the brief's steps one-to-one (17 nodes, 6 skills). It was reviewed against measured cost and value, and every node that did not justify itself was merged or removed. Rule used: a node must do something no other node can, or be a quality gate.

Removed / changed	Replaced by	Why
load_memory || retrieve_cases	memory loaded in intake, then retrieve	Memory load is a millisecond SQLite read, so parallelism bought nothing; loading it first lets remembered facts improve the search query.
select_skills	merged into triage	It only validated names triage had already chosen.
focused_retrieval	the agent's filtered search tool	Duplicated model-decided retrieval and inflated every prompt.
assess (resolve / clarify / escalate)	ask_customer and escalate_ticket tools	The investigating model already holds the evidence; a separate decision call cost ~3K tokens per ticket against an 8K/min limit.
ask_clarification node	interrupt inside the ask_customer tool	Same pause/resume semantics, no extra node; the answer returns as a tool result.
draft_resolution + escalate	respond (one schema)	Same output structure with an escalation flag.
index_case	merged into finalize	Both are deterministic persistence.
notes call after the last tool round	tools → respond when budget spent	Saved one 120b call (~3.5K tokens) per ticket.
skills ticket_triage, customer_response	moved into the triage / respond prompts	Loaded on every run, so they were prompts, not skills.

Kept on purpose: evaluate (the brief's evidence check with retry), review (prevents unchecked LLM output from entering the knowledge base) and the finalize || update_memory fan-out (two independent side effects, one of them an LLM call). The support-lead feature followed the same rule and added **no node**: an amend decision is handled inside review and routed to respond, and the normal evidence check and review run again.

7. State and memory

Graph state

Field	Reducer	Purpose
messages	add_messages	Investigation scratchpad and conversation: ticket, tool calls/results, clarifications, judge and lead feedback.
evidence	custom merge_evidence	Evidence pool keyed by case id; repeated retrievals merge (best similarity kept, every retrieval path recorded). Also holds the lead's diagnosis as LEAD-<case_id>.
retrieval_log, clarifications	operator.add	Audit trail of every search and every question/answer.
classification, skills, label_votes	last value	Triage output, loaded skills, neighbour label distribution.
customer_memory, customer_history	last value	Loaded per turn by intake.
tool_rounds, escalation_requested, investigation_notes	last value	Tool budget, escalation flag, notes for the writer.
lead_guidance	last value	The lead's root cause, solution and note (who, when); kept across follow-up turns and saved to the case row at once.
draft, evaluation, review, final	last value	Answer; evidence-check result (also counts failed attempts for the retry loop); review decision (accept / amend / reject); final structured output (incl. the index result).

Field	Reducer	Purpose
memory_update	last value	What the parallel memory node saved or forgot.

State fields are not sent to the LLM as such: each node builds its prompt from a trimmed view of them (evidence digest, memory digest, the last 40 messages / 60K characters).

Short-term memory

The SqliteSaver checkpointer writes the full state to checkpoints.db after every node, keyed by thread_id. At the start of a run LangGraph loads the latest checkpoint of that thread (a local read, not an API call); a new ticket gets a new thread and starts empty. This gives interrupt/resume across process restarts, retry from the last checkpoint, replay, and multi-turn follow-ups: a follow-up re-enters at intake, keeps messages, evidence, classification, skills, clarifications and lead guidance, skips retrieve and triage, and resets only per-turn fields (draft, evaluation, budgets). A follow-up sent while a clarification question is open is treated as its answer.

Long-term memory

Aspect	Technique
Storage	LangGraph SqliteStore in memory.db, namespace ("customers", customer_id, "facts"), one item per fact key.
What to remember	LLM step with an explicit policy: durable customer-level facts only (product, environment, plan, integrations, preferences, recurring issues, constraints). Not stored: one-off details, resolution steps, speculation.
Safety	Deterministic filter rejects card-like numbers, password/PIN/CVV/secret/token/API-key mentions, IBAN- and SSN-like patterns, and values over 300 chars.
Freshness	Upsert by key (changed facts overwrite); obsolete keys explicitly forgotten; ≤4 new facts per case; hard cap of 10 per customer (oldest dropped).
Use	Loaded at intake; enriches the retrieval query; shown to triage, investigate and respond; available via get_customer_history; viewable and deletable through the memory API.

Customer history

Every case lives in the SQLite cases table from the moment it arrives (open → awaiting_customer → resolved / escalated), so open and new cases count as customer history immediately. History cases are also citable evidence, which is what justifies a "recurring issue → escalate" decision.

Storage map

Store	Type	Holds	Used by
Qdrant collection support_cases	Vector DB: dense + BM25 vectors, keyword payload indexes	Historical and accepted live cases: vectors of subject + body, payload with answer, labels, tags, source and lead fields	retrieve, search_similar_tickets, finalize (upsert), reset_live_cases.py
data/app.db	SQLite (sqlite3, WAL)	historical_tickets · cases (full lifecycle, lead_guidance, review, indexed flag) · events · llm_usage_by_key	get_ticket_by_id, category statistics, label vocabulary, intake / finalize, customer history, replay, /api/health
data/checkpoints.db	SQLite via LangGraph SqliteSaver	Full graph state per thread_id after every node, pending interrupts	every run: resume, follow-up, retry, lead-solution on finished cases
data/memory.db	SQLite via LangGraph SqliteStore	Customer facts	intake, update_memory, get_customer_history, memory API

Both LangGraph stores are on disk (not InMemorySaver / InMemoryStore), so paused cases and customer memory survive a restart. Historical tickets are deliberately in Qdrant and in app.db: Qdrant answers "which cases are similar", SQLite answers exact look-ups and counts.

8. Case-based RAG

Technique	Detail
Hybrid retrieval	Dense (bge-small, cosine) and BM25 (sparse, IDF) prefetches fused with Reciprocal Rank Fusion inside Qdrant's Query API. BM25 catches error codes and product names; dense catches paraphrases.
Interpretable scores	RRF scores are rank-based, so the cosine similarity of every hit is recomputed from the returned dense vector for display and confidence.
Near-duplicate removal	Hits with the same subject/body signature are dropped (the dataset contains many near-copies).
Memory-enriched query	Ticket + clarifications + follow-up + remembered product/environment facts, so a vague "it is broken again" still finds cases about the customer's system.
Metadata filters	queue / type / priority / tag / source / exclude-ids. The initial retrieval is unfiltered because it runs before triage; filtered searches are chosen by the model through search_similar_tickets (often with the triaged labels).
Filter relaxation	If a filtered search returns nothing, it is retried once without filters.
Strict-mode indexes	Qdrant Cloud rejects filters on non-indexed fields, so keyword payload indexes exist on queue, type, priority, tags, source, case_id (created at startup if missing).
Trusted new cases	A dedicated source=live search (only when live cases exist) so a few new resolutions are not out-ranked by 23K historical near-duplicates.
Lead-verified preference	Lead-verified cases get +0.10 in evidence ranking; the lead's own diagnosis of this case (LEAD-…) always ranks first; clip_answer() never cuts the lead's block when a resolution is shortened for a prompt.
Neighbour label votes	Similarity-weighted label distribution of retrieved cases, used as a prior for triage.
Evidence pool	All retrieval paths merge into one pool; the model sees a compact digest and full records on demand (get_ticket_by_id, also for LEAD- ids).
No repeated evidence	search_similar_tickets excludes case ids already in the pool (Qdrant must_not), so a tool search only ever adds new cases.
Multi-case comparison	Prompts require consensus vs conflicts; respond reports conflicts; the evidence check requires ≥2 valid citations for a resolution (or the lead's diagnosis).
No leakage in evaluation	Evaluation tickets were held out at ingestion and never indexed.

9. Tools and skills

Tools (the model decides if, when and which)

Tool	Purpose	State effect
search_similar_tickets	Targeted hybrid search with optional queue / type / priority / tag filters (k ≤ 6). Cases already in the evidence pool are excluded in Qdrant, so it returns only new evidence; results flag new and lead-verified cases.	adds evidence + retrieval log
get_ticket_by_id	Full record of a historical (HT-…) or live (CASE-…) case, or a lead's diagnosis (LEAD-CASE-…), before relying on it.	adds evidence
get_customer_history	Previous / open cases and remembered facts.	–
get_category_statistics	Volume, type/priority distribution and top tags for a queue, tag or type.	–
update_classification	Correct triage when evidence disagrees, with a reason.	updates classification
escalate_ticket	Flag escalation with reason, target team, urgency.	respond produces an escalation
ask_customer	One focused clarification question; pauses the graph until answered. No fixed limit: repeats, asks after "no more details" and requests for secrets are rejected.	adds Q/A; answer returned as tool result

Tool techniques

- State-changing tools return LangGraph Command updates (evidence, classification, escalation flag, clarifications) together with the ToolMessage.
- Injected arguments (InjectedToolCallId, InjectedState, InjectedStore) are hidden from the model's schema.
- Model inputs are normalised ("null"/"any" → none; fuzzy match to valid labels with difflib).
- interrupt() inside a tool: the ToolNode pauses, and on resume the tool returns the customer's answer.
- Tool errors become ToolMessages (handle_tool_errors), so the model can recover instead of crashing the run.

Skills (progressive disclosure)

A module is a skill only if it is loaded conditionally. Only the catalog (name + description) is shown when choosing; the full instructions enter the prompt after selection.

Skill	Loaded when	Key guidance
incident_investigation	triage picks it (Incident/Problem, technical queues)	hypothesis → targeted search → open records → compare fixes; security/outage escalation
request_fulfillment	triage picks it (Request/Change, sales/HR/general)	identify what is requested and prerequisites; ask rather than guess on ambiguity
billing_and_accounts	triage picks it (billing, returns)	never promise refunds/timelines not in evidence; never ask for card numbers
escalation	high priority, repeat Incident/Problem, or escalation flagged	criteria + hand-off format + holding reply; used by investigate and respond

10. Response, evidence check and confidence

respond

One structured output (pydantic) for both outcomes: analysis, conflicts, resolution (customer reply), internal_actions, evidence_used (case id + how used), escalation_required, escalation_reason, target_team, urgency, handoff_summary, self_confidence. The **grounding rule**: every step or promise must appear in at least one cited case; no generic steps from the model's own knowledge; if the cases only ask for details or describe a standard process, the reply does the same.

Support-lead rule. When the lead gave a diagnosis or rejection feedback, respond receives it verbatim. It overrides conflicting cases and counts as support for the grounding rule; every point the lead made becomes a regular step, never "optional", and investigation notes that doubt it are ignored. The "mark single-source steps optional" rule applies to historical cases only.

evaluate

Check	Type	Rule
cites_existing_evidence	rule	every cited id exists in the evidence pool or customer history
compares_multiple_cases	rule	≥ 2 valid citations, or the lead's diagnosis cited (resolutions only)
has_customer_reply	rule	reply ≥ 60 characters
classification_valid	rule	queue, type and priority set
has_escalation_reason, valid_target_team	rule	escalations only (no LLM judge needed: a holding reply makes no factual claims)
groundedness, addresses_issue	LLM judge (20b)	share of substantive claims supported by cited cases, customer info or support-lead input; information requests and process statements need no support

Pass = judge says grounded **and** groundedness ≥ 0.7 **and** all rule checks pass. The judge runs on a different model from the writer to reduce self-preference.

Confidence score

```
confidence = 0.40 · groundedness
           + 0.20 · addresses_issue
           + 0.20 · retrieval_strength   (1.0 if the lead's diagnosis is cited; else mean cosine of the
                                          top-3 cited cases, mapped 0.55..0.90 → 0..1)
           + 0.10 · label_agreement      (share of cited cases in the predicted queue)
           + 0.10 · model self-confidence
           − 0.05 per unresolved conflict (max 0.15)  − 0.15 if any citation is invalid   → clamped to 0..1
```

Retry and escalation policy

- First failure: back to investigate with the judge's feedback and suggested query, one tool round, then straight to respond.
- Second failure: forced escalation ("evidence check failed"), never an unsupported answer.
- review: a support lead accepts, rejects with feedback, or amends with a diagnosis (auto-accept for demos/evals).

11. Continuously growing case knowledge

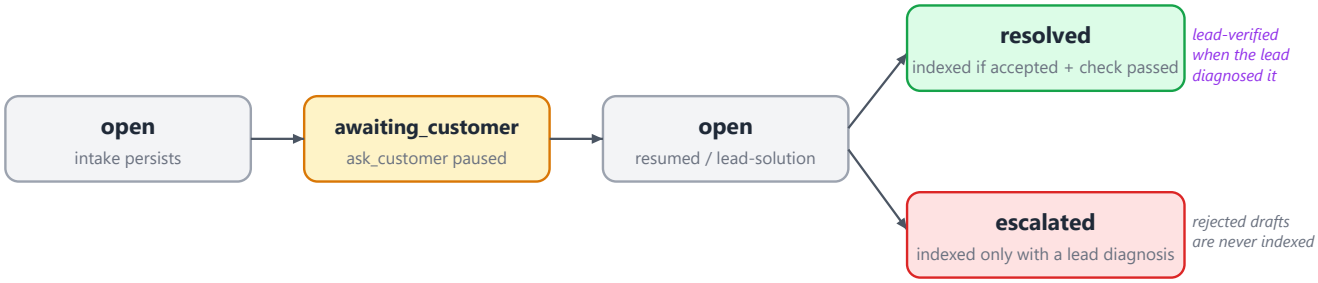


Figure 3 · Case lifecycle in the SQLite cases table, and when a case becomes searchable knowledge.

- Every query is stored at intake as an **open** case; status moves through awaiting_customer, open, then resolved or escalated.
- finalize stores classification, evidence, analysis, resolution, escalation, confidence, evaluation, clarifications, review and lead guidance.
- The case is indexed into Qdrant (source = live) **only if** the reviewer accepted it and either it is a resolution that passed the evidence check, or it carries a support-lead diagnosis (even an escalation). Escalated cases without a diagnosis and rejected drafts stay as history but are not trusted knowledge (prevents knowledge-base poisoning).
- Future tickets find these cases through normal hybrid search and the trusted-new-cases lookup; the UI labels them *new case* or *lead-verified*.
- **scripts/reset_live_cases.py** removes live points from Qdrant (all, or one --case-id) and marks the rows indexed=0; historical tickets, memory and checkpoints are untouched. Run it before a demo so rehearsal answers do not change the next run.

Support-lead knowledge (lead-verified cases)

The historical base cannot contain fixes nobody has written down yet. When the reviewing lead knows the real cause, that knowledge is used for this customer and for every similar ticket after it.

Step	What happens
Input	The review card has Root cause and Solution fields with an **Apply my diagnosis** button (action amend). A diagnosis typed before clicking Accept is applied too, never dropped; feedback given with Accept is saved as a review note.
Authoritative evidence	review stores the diagnosis in lead_guidance (and in the case row right away) and adds it to the evidence pool as LEAD-<case_id>, ranked first. The graph goes review → respond; no new investigation is needed.

Step	What happens
Grounded on the lead	respond and the judge get the diagnosis plus every rejection comment verbatim. Lead input overrides conflicting cases and counts as support; citing LEAD-… satisfies the multiple-cases check. The rewritten reply returns to review; Accept finalises it.
Indexed as lead-verified	finalize upserts the case with lead_verified=true, root_cause and lead_solution in the payload. Its answer starts with "[SUPPORT-LEAD VERIFIED] Root cause … Verified solution …", followed by "Reply sent: …" (for an escalation only the lead's part).
Reused, never cut	+0.10 ranking over equally similar historical cases; prompts label them LEAD-VERIFIED CASE; investigate and respond prefer their fix over conflicting older cases; clip_answer() keeps the lead's block whole and shortens only the reply text.
Never downgraded	The lead's points need no backing from other cases. investigate lists each as required; respond turns each into a regular step and ignores notes that doubt them. A note like "X might be the issue, check it too" becomes a normal step next to the diagnosis.
Finished or escalated cases	POST /api/threads/{id}/lead-solution (also under the result in the UI) applies a diagnosis afterwards: the state is updated as if review had decided amend, then respond → evaluate → review → finalize. An escalation becomes a resolution built on the lead's fix and is indexed as lead-verified.

12. LLM layer, usage accounting and limits

The Groq account is on a paid plan, so the app enforces no Groq limits: it never waits for, trims or refuses a call because of token or request counts. On the earlier free tier (8K TPM, 200K TPD per model) a per-model token bucket mirrored Groq's refill and daily guards failed fast; both were removed after the upgrade. The token trims were kept because fewer tokens also means faster answers.

Service	Limit	Handling
Groq (paid plan)	reported per key and model in x-ratelimit-* headers	Usage recorded per day, API key (SHA-256 fingerprint) and model in app.db; limits shown in /api/health, never enforced; the Groq SDK retries 429/5xx (max_retries 3, honours retry-after); a remaining failure can be retried from the checkpoint.
Single request	model context window	Generous output caps (4,096 main / 2,048 fast) only against runaway replies; no prompt shrinking; history keeps the last 40 messages / 60K characters.
Qdrant Cloud (free, strict mode)	no per-collection RPS cap; unindexed filters rejected; 1 GB RAM	Payload indexes; client-side throttle 5 req/s with exponential backoff; uploads batched 128 points/request; collection-exists and counts cached.

Token-efficiency techniques

Technique	Effect
Role split: main model for investigation and the final answer; fast model for triage, judge, memory	Cheaper, faster steps; judge independent from writer
Reasoning effort low by default (configurable)	Lower latency; reasoning tokens count as output
Removed the separate decide (assess) node	≈ 3K tokens saved per ticket
Skip the notes-only call when the tool budget is spent	≈ 3.5K tokens and one round trip saved per ticket
Several tool calls per round, 2-round budget	Fewer full-context re-sends
Static-first prompts; stable evidence block across rounds	Enables Groq prompt caching (cached tokens observed)
Agent starts from the evidence pool; searches only when it is weak	Skips a redundant search round (was the first action in 27 of 29 logged runs)
Follow-up turns reuse evidence + classification from state	No re-retrieval and no re-triage on a reopened case
Grounding rule + 1 retry instead of 2	Stopped ~27K-token failing tickets caused by generic, unsupported steps

Technique	Effect
Search returns only cases not yet in the evidence pool	No case is sent twice in one prompt (was 4 of 5 in a measured run)
Top 5 cases per prompt instead of 8 near-copies; triage gets labels + subjects only	≈ 1.6K tokens per ticket
Short tool descriptions; compact label shares instead of JSON	≈ 0.5K tokens per ticket
Escalation skill split: criteria while investigating, hand-off format only when escalating	≈ 0.3K tokens per ticket
Lead amend goes review → respond, not back to investigate	No investigation rounds when the lead already gave the answer
Structured-output slips repaired locally before any retry	Avoids a second call (≈ 0.6K tokens) when the reply is only mis-formatted

Structured output

Every decision node returns a pydantic object via tool-style function calling. If the model mis-formats, the reply is first repaired locally (tool-call args, JSON in the text, or Groq's failed_generation); only if that fails is the call retried once in JSON mode with the schema attached. API errors (rate limit, network, auth) are surfaced instead of being retried in another format. Each call emits an llm_call event (input, output, cached and counted tokens) shown per node in the UI.

Measured cost per ticket

Call	Model	Typical tokens
triage	fast	0.82–0.92K in / 0.12–0.19K out (was 1.33K in)
investigate (per round)	main	1.9–2.8K in / 0.05–0.5K out (was 2.6–3.6K in)
respond	main	1.3–2.0K in / 0.3–0.5K out (was 2.4K in)
evaluate (judge)	fast	0.9–1.5K in / 0.06–0.3K out
update_memory	fast	0.4–0.5K in / 0.05–0.23K out (one call)
Resolved ticket total		≈ 7.1–10.6K tokens in 5–6 calls (up to ~15K with an evidence-check retry); each extra clarification round adds ≈ 2.2K; a lead amend adds one respond and one judge call

13. Evaluation (LangSmith)

`python -m evals.run_evals --n 3` uploads held-out tickets as the LangSmith dataset support-agent-heldout and runs langsmith.evaluate() on the full agent (clarifications auto-answered, review auto-accepted). Without a LangSmith key the same evaluators run locally and write data/eval_report.json. LANGSMITH_TRACING=true traces every graph run.

Evaluator	Type	Measures
classification_accuracy	rule	queue / type / priority exact match + tag Jaccard vs the reference
retrieval_queue_precision	rule	share of retrieved cases in the reference queue
retrieval_relevance	LLM judge	relevance of the top-5 retrieved cases to the ticket
response_quality	LLM judge	agent reply vs the dataset's reference reply (correctness, helpfulness, tone)
groundedness	LLM judge	reply supported by the cited cases
tool_usage	rule	tools used, only valid tools, within budget
agent_behavior	rule	all output fields present, confidence in range, escalation reason, evidence check ran, case persisted, ≥2 cases

14. API, streaming and frontend

Method	Path	Purpose
POST	/api/tickets	New ticket {customer_id, subject, body, auto_accept} → SSE stream
POST	/api/threads/{id}/resume	{answer} for a clarification, or {action: accept / amend / reject, feedback, root_cause, solution} for review → SSE
POST	/api/threads/{id}/lead-solution	{root_cause, solution, note}: the lead's diagnosis at review or for a finished / escalated case → SSE (404 unknown thread, 409 case busy)
POST	/api/threads/{id}/followup	New turn on the same thread (or the answer to an open question) → SSE
POST	/api/threads/{id}/retry	Resume from the last checkpoint after an error
GET	/api/threads/{id}	State snapshot + persisted event timeline
GET	/api/cases · /api/tickets/{id} · /api/customers/{id}/memory	Case list · full ticket or case · memory and history
DELETE	/api/customers/{id}/memory	Forget all facts of a customer
GET	/api/graph · /api/health · /api/samples	Graph topology · status incl. Groq usage and vector counts · demo scenarios

Event types streamed: run_started, node (with summary and data), llm_call, llm_note, interrupt, completed, error. Every event is stored in SQLite (events table) so any case can be replayed later.

Web UI features

- Live workflow graph drawn from the compiled graph; visited, last-step and waiting nodes highlighted; loop counts (e.g. investigate ×2).
- Live trace with per-node summaries, token counts per call and expandable details (tool calls, results, retrieved cases).
- Clarification panel (question + answer box) and review panel (draft, evidence-check result, the lead's Root cause / Solution fields, Accept · Apply my diagnosis · Reject & re-investigate).
- Result card: status, confidence ring, classification chips, reply, internal actions, escalation, the lead's diagnosis, the index result, evidence table with similarity bars and *new case* / *lead-verified* / *support lead* labels, evidence-check metrics.
- Under a finished case: "Resolve this escalation with the lead's solution" (or "Correct this case"), which calls the lead-solution endpoint.
- Customer memory panel, case list with replay, ticket viewer (also for LEAD- ids), follow-up box, demo-scenario loader with fresh customer ids, header with Groq usage today and Qdrant counts.

15. Techniques catalogue

Every technique used in the implementation, grouped by area.

Area	Techniques
Agent design	ReAct loop inside a verify-and-commit pipeline · Model-decided tool use (no fixed sequence) · Human-in-the-loop via tool-level interrupt · Self-verification with an independent LLM judge · Escalation as a safe fallback · Human trust gate before knowledge grows · Expert input as authoritative evidence (lead-verified cases) · Lean graph: every node justified by cost/value
LangGraph	StateGraph with typed state and reducers (add_messages, custom merge, operator.add) · Conditional edges incl. list return for parallel fan-out · ToolNode with Command-returning tools · InjectedState / InjectedStore / InjectedToolCallId · interrupt() + Command(resume) · update_state(as_node="review") to continue a finished thread · SqliteSaver checkpointer, SqliteStore long-term memory · custom stream mode via get_stream_writer · Thread-based multi-turn with per-turn state reset · recursion limit and tool budgets
Retrieval	Hybrid dense + BM25 with server-side RRF · Named vectors in one collection · Problem-only embedding · Memory-enriched queries · Metadata filters + relaxation fallback · Strict-mode payload indexes · Near-duplicate removal · Cosine re-scoring from returned vectors · Neighbour label voting · Trusted-new-case lookup · Lead-verified ranking boost · Evidence pool merging · Exclusion of known ids · Hold-out split against leakage
Prompting and LLM	Role-based model split · Pydantic structured output with JSON-mode fallback · Grounding rule against outside knowledge · Lead-input rules (overrides cases, never downgraded) · Lead-safe answer clipping · Hyphen normalisation of cited ids · Progressive-disclosure skills · Static-first prompts for caching · History trimming that never orphans tool results · Low reasoning effort + output caps
Memory	LLM-curated memory policy · Deterministic PII / secret filter · Upsert-by-key and explicit forgetting · Per-customer cap · Case history from the structured store, citable as evidence
Reliability and limits	Measured limits before testing · Per-key token accounting (paid plan: no client limits) · Plan limits read from response headers · SDK retries on 429/5xx, checkpoint retry · Local repair of structured-output slips · Persisted daily usage · Qdrant throttle with exponential backoff · Batched, idempotent uuid5 upserts · Cached counts/exists · Resumable local → cloud vector sync · Demo reset of live cases
Data engineering	Language filtering, cleaning, exact de-duplication · Length-sorted embedding batches · Dense-text truncation, full-text BM25 · Local CPU embeddings (no embedding API limits)
Observability and UX	SSE streaming of node and LLM-call events · Persisted event log and replay · Graph diagram generated from the compiled graph · Per-node token accounting · Optional LangSmith tracing
Evaluation	LangSmith datasets and experiments · Rule evaluators + LLM-as-judge evaluators · Trajectory capture (nodes, tool calls, retrieved cases) · Local fallback report
Security and config	Secrets only in a gitignored .env · Env merging with precedence and aliases · Duplicate-key warning for .env · No secrets or payment data in memory · Fresh demo customers to avoid state leakage

16. Verified demo runs

Run with scripts/demo.py on the Groq free tier and Qdrant Cloud (auto-accept review, scripted customer answers).

Required path	Result
Normal resolution	Duplicate-charge ticket → resolved, groundedness 1.0, confidence 0.996; 5 consistent cases cited; indexed as trusted. Memory correctly stored nothing (no durable facts; the customer's name was not kept).
Clarification + resume	Vague "Not working" → the agent called ask_customer ("which product or service, any error messages…"); the graph paused, resumed with the answer, searched again and resolved at groundedness 1.0, confidence 0.76.
Long-term memory reuse	Ticket 1 stored 4 facts (Enterprise plan, Azure, EMR integration module, telemedicine platform). A separate conversation, "Sync delays are back", loaded them and needed no clarification.
Retrieval of a newly resolved case	In that second conversation the top evidence was the case resolved minutes earlier (similarity 0.82, labelled new case) and it was cited in the answer.
Support lead teaches a new fix (scenario 5, added later)	Available with `python scripts/demo.py --only lead_knowledge`. The lead's diagnosis (Premium dashboard over its QuickBooks RPS limit; shared cache + 15-minute refresh, or a quota increase) is applied at review and the case is indexed as lead-verified; a similar ticket from another customer should retrieve and cite it. The script prints a CHECK line for each of these. Not part of the original verified runs above.

Behaviour observed during testing

- Memory and history change decisions correctly: with reused customer ids the agent skipped clarification (memory already held the answer) and escalated recurring issues. The demo therefore uses fresh customer ids per run (--reuse-customers to disable).
- Accepted demo answers are indexed and change the next run of the same scenario (a vague ticket can be answered from its earlier clarified copy instead of asking). scripts/reset_live_cases.py --yes restores a clean knowledge base before a demo.
- The evidence check works: before the grounding rule, the 120b model added generic steps not in the dataset (e.g. "re-authorize the integration"); the judge rejected them and the agent escalated rather than answer without support.
- Rate limiting (free tier, before the paid plan): the UI and the demo showed TPM waits, and when both daily budgets ran out, runs stopped with a clear message instead of looping. That limiter has since been removed.
- Cloud copy: 23,608 points copied in 72 s at ~330 points/s with zero retries under the 5 req/s throttle.

17. How to run

```
# backend + frontend (one command; open http://localhost:8000)
cd D:\Agent_Latest\backend
D:\Agent_Latest\.venv\Scripts\python.exe -m uvicorn app.api:app --port 8000

# scripted demo of the four required paths + the lead scenario (fresh customers each run)
D:\Agent_Latest\.venv\Scripts\python.exe scripts\demo.py
D:\Agent_Latest\.venv\Scripts\python.exe scripts\demo.py --only lead_knowledge

# before a live demo: drop resolutions indexed by rehearsals (list first, then remove)
D:\Agent_Latest\.venv\Scripts\python.exe scripts\reset_live_cases.py
D:\Agent_Latest\.venv\Scripts\python.exe scripts\reset_live_cases.py --yes

# evaluations (LangSmith if LANGSMITH_API_KEY is set, otherwise local report)
D:\Agent_Latest\.venv\Scripts\python.exe -m evals.run_evals --n 3

# rebuild the index from scratch (optional)
D:\Agent_Latest\.venv\Scripts\python.exe scripts\ingest.py
D:\Agent_Latest\.venv\Scripts\python.exe scripts\sync_qdrant.py

# regenerate this document
D:\Agent_Latest\.venv\Scripts\python.exe D:\Agent_Latest\docs\build_design_pdf.py
```

18. Limitations and next steps

- **Dataset nature**: synthetic tickets; many historical answers ask for more details rather than solving, so grounded replies often follow that pattern. Lead-verified cases are the way to add real fixes.
- **No customer ids in the dataset**: customer history exists only for new tickets.
- **Lead input is trusted as authoritative** and there is no reviewer authentication yet: anyone with UI access can act as the support lead, and a wrong diagnosis would be indexed as lead-verified (it can be removed with reset_live_cases.py --case-id).
- **Groq limits**: none enforced by the app (paid plan); the plan's own limits are only reported. On the free tier the removed limiter would be needed again.
- **LLM-as-judge bias**: mitigated by rule checks and a different judge model, not eliminated.
- **Concurrency**: sync graph with SQLite is fine locally, and embedded Qdrant is single-process; an async graph with a Postgres checkpointer and a Qdrant server would be the production path.
- **Next steps**: cross-encoder re-ranking, a post-triage filtered retrieval pass, clustering resolutions into approaches, confidence calibration on the eval set, reviewer authentication and an audit trail for lead edits.

Appendix · Configuration reference

Variable	Default	Meaning
GROQ_API_KEY	–	Groq key (required)
GROQ_MODEL / LLM_MAIN	openai/gpt-oss-120b	investigation and response model
GROQ_FAST_MODEL / LLM_FAST	openai/gpt-oss-20b	triage, judge, memory model
GROQ_MAX_OUT_MAIN · GROQ_MAX_OUT_FAST	4096 · 2048	output caps (against runaway replies only)
GROQ_REASONING_EFFORT	low	gpt-oss reasoning effort
QDRANT_URL · QDRANT_API_KEY · QDRANT_COLLECTION	– · – · support_cases	Qdrant server (empty URL = embedded local mode)
QDRANT_PATH	backend/data/qdrant	embedded store location
QDRANT_MAX_RPS · QDRANT_UPLOAD_BATCH	5 · 128	Qdrant throttle and batch size
DENSE_MODEL · SPARSE_MODEL	BAAI/bge-small-en-v1.5 · Qdrant/bm25	local embedding models
RETRIEVAL_K · MAX_TOOL_ROUNDS	6 · 2	initial retrieval size (k+2 fetched) · tool rounds
MAX_EVAL_RETRIES · MAX_MEMORY_FACTS	1 · 10	quality loop, memory cap
LANGSMITH_API_KEY · LANGSMITH_TRACING · LANGSMITH_PROJECT	– · false · support-investigation-agent	evaluation and tracing